

Making `std::owner_less` more flexible

- - [Introduction](#)
 - [Motivation and Scope](#)
 - [Design Decisions](#)
 - [Impact On The Standard](#)
 - [Technical Specification](#)
 - [Implementation Experience](#)
 - [Acknowledgements](#)

Introduction

The class template `std::owner_less` is inconvenient to use for anything but the simplest cases:

```
shared_ptr<int> sp1;
shared_ptr<void> sp2;
shared_ptr<long> sp3;
weak_ptr<int> wp1;

owner_less<shared_ptr<int>> cmp;
cmp(sp1, sp2); // error, doesn't compile
cmp(sp1, wp1);
cmp(sp1, sp3); // error, doesn't compile
cmp(wp1, sp1);
cmp(wp1, wp1); // error, doesn't compile
```

Motivation and Scope

The first error in the example in the introduction can be avoided by using `owner_less<shared_ptr<void>>` but the initial attempt to do so still fails because `sp1` can be converted to both `shared_ptr<void>` and `weak_ptr<void>`:

```
owner_less<shared_ptr<void>> cmpv;
cmpv(sp1, sp2); // error, ambiguous conversion
```

To make it compile it's necessary to explicitly convert the `shared_ptr<int>` to the right argument type:

```
owner_less<shared_ptr<void>> cmpv;
cmpv(shared_ptr<void>(sp1), sp2);
```

Not only is this more verbose and less clear, but the conversion creates a temporary, incrementing and decrementing the reference count.

`shared_ptr::owner_before` and `weak_ptr::owner_before` are function templates that support mixed comparisons between `shared_ptr<A>` and `weak_ptr<B>`, but `owner_less` only supports comparing `shared_ptr<A>` and `weak_ptr<A>`. Mixing pointers to different types either causes implicit conversions (and reference-count updates) or just fails to compile. This is an unnecessary restriction, because the `shared_ptr` aliasing constructor means that objects which share ownership can store completely unrelated types of pointer.

Even if we don't care about mixing different types, the three `owner_less::operator()` overloads only support three out of four combinations of argument types, and the [Adaptable Binary Function](#) typedefs `first_argument_type` and `second_argument_type` only describe the argument types for one of those three overloads.

The proposed addition removes the typedefs, as they are unnecessary since C++11, and allows the example in the introduction to handle all the comparisons without creating any temporaries:

```
owner_less<> cmp;
cmp(sp1, sp2); // no temporary created
cmp(sp1, wp1);
cmp(sp1, sp3); // ok
cmp(wp1, sp1);
```

```
cmp(wp1, wp1);    // ok
```

Design Decisions

The proposed addition is directly inspired by [N3421](#) and so is not novel.

There is no need to constrain the proposed member function templates, because `owner_before` is unconstrained.

The member function templates simply return `bool` rather than using `decltype` to detect the result of `x.owner_before(y)`. Although this means that in [N3421](#) terms the proposed specialization is not completely "transparent" there is no need to use `decltype` because `bool` is always the right return type. By defining the nested `is_transparent` typedef the following is supported:

```
set<shared_ptr<X>, owner_less<>> s;  
shared_ptr<void> key = get_key();  
auto iter = s.find(key);
```

It might makes sense to alter the existing `owner_less<shared_ptr<T>>` and `owner_less<weak_ptr<T>>` partial specializations to add a fourth overload that takes two `weak_ptr`s or two `shared_ptr`s respectively, and also to remove the nested typedefs. The fourth overload seems unnecessary if this proposal is accepted, because it would still be simpler to use the new `owner_less<void>` specialization. Removing the typedefs is a potentially breaking change so is left for another proposal to do later if that change is wanted.

Impact On The Standard

This is a pure library extension. In C++ today `owner_less<void>` is an incomplete type, so defining it should not cause problems for any correct programs.

Technical Specification

In [util.smartptr.ownerless] p1 add a default template argument of type `void`,

```
template<class T_ = void> struct owner_less;
```

And after the two partial specializations add:

```
template<> struct owner_less<void> {  
    template<class T, class U>  
        bool operator()(shared_ptr<T> const&, shared_ptr<U> const&) const;  
    template<class T, class U>  
        bool operator()(shared_ptr<T> const&, weak_ptr<U> const&) const;  
    template<class T, class U>  
        bool operator()(weak_ptr<T> const&, shared_ptr<U> const&) const;  
    template<class T, class U>  
        bool operator()(weak_ptr<T> const&, weak_ptr<U> const&) const;  
  
    typedef unspecified is_transparent;  
};
```

Implementation Experience

The implementation is trivial and has been tested in the GNU libstdc++ library.

Acknowledgements

Thanks to Marshal Clow for comments on the proposal.